

Relatório – Sistema de Vendas e Fidelidade

Cafeteria Geek “Byte & Brew”

1. Introdução

Este trabalho tem como objetivo aplicar os conceitos fundamentais da Programação Orientada a Objetos (POO) por meio do desenvolvimento de um sistema de vendas e fidelidade para a cafeteria temática **Byte & Brew**.

O sistema simula o funcionamento de uma cafeteria geek, contemplando o gerenciamento de produtos, clientes, pedidos, controle de estoque, aplicação de promoções e um programa de fidelidade baseado em XP (Experiência).

Durante o desenvolvimento foram aplicados conceitos como herança, polimorfismo, encapsulamento, abstração, interfaces, exceções personalizadas e organização em pacotes.

2. Objetivos do Sistema

O sistema foi desenvolvido com os seguintes objetivos:

- Gerenciar produtos do cardápio (comidas e bebidas)
- Controlar o estoque dos produtos
- Cadastrar e gerenciar clientes
- Registrar pedidos de compra
- Aplicar regras de fidelidade (XP)
- Implementar promoções em eventos especiais
- Garantir consistência com validações e exceções

3. Estrutura do Projeto

O projeto foi organizado em pacotes, seguindo boas práticas de desenvolvimento:

`br.edu.cafeteria`

```
|— app
|— modelo
|— servico
|— excecao
```

Responsabilidade dos pacotes:

- **app**: classe principal (execução do sistema)
- **modelo**: entidades do sistema (Produto, Cliente, Pedido etc.)
- **servico**: regras de negócio e promoções
- **excecao**: tratamento de erros personalizados

4. Modelo de Classes (Visão Geral)

O sistema é composto pelos seguintes principais componentes:

- Produto (abstrato)
 - Comida
 - Bebida
- Cliente (abstrato)
 - Cliente Standard
 - Cliente VIP
- Pedido
- ItemPedido
- Atendente
- Interface Promocional
- Promoção de Evento Geek

5. Diagramas e Relacionamentos

5.1 Herança

O sistema utiliza herança para especializar entidades:

- Produto → Comida / Bebida
- Cliente → ClienteStandard / ClienteVIP

5.2 Associação entre Classes

- Um Pedido possui vários ItemPedido
- Um ItemPedido referencia um Produto
- Um Pedido está associado a um Cliente (ou cliente casual)
- Um Pedido está associado a um Atendente

5.3 Polimorfismo

Foi aplicado de diferentes formas:

Polimorfismo por inclusão

Produtos tratados de forma genérica dentro de pedidos.

Polimorfismo por sobrescrita

O método `calcularXP()` é sobrescrito nas classes:

- `ClienteStandard`
- `ClienteVIP`

Polimorfismo por interface

A interface `Promocional` permite diferentes tipos de promoção.

5.4 Interface

Foi criada a interface:

`Promocional`

Responsável por definir o comportamento de descontos em promoções.

A classe `PromocaoEventoGeek` implementa essa interface.

6. Regras de Negócio

6.1 Controle de Estoque

Um produto não pode ser vendido se não houver quantidade suficiente em estoque.

Isso é garantido pela exceção:

- `EstoqueInsuficienteException`

6.2 Programa de Fidelidade (XP)

O sistema possui dois tipos de clientes:

Cliente Standard

- Ganha 1 XP por R\$ 1,00 gasto

Cliente VIP

- Ganha 2 XP por R\$ 1,00 gasto
- Pode pagar compras com XP
- Conversão: 10 XP = R\$ 1,00

6.3 Resgate de XP

O Cliente VIP pode utilizar XP para pagar compras, desde que possua saldo suficiente.

Caso contrário, é lançada a exceção:

- `PontosInsuficientesException`

6.4 Promoções

O sistema suporta promoções através da interface `Promocional`.

Durante eventos especiais, é aplicada:

- 10% de desconto em bebidas

7. Tratamento de Exceções

Foram criadas exceções personalizadas para garantir consistência do sistema:

`EstoqueInsuficienteException`

Lançada quando o estoque não é suficiente para atender a venda.

`PontosInsuficientesException`

Lançada quando o cliente não possui XP suficiente para pagamento.

Essas exceções são do tipo **checked**, garantindo tratamento obrigatório.

8. Conceitos de Programação Orientada a Objetos

8.1 Encapsulamento

Todos os atributos das classes são privados, com acesso controlado por métodos.

8.2 Herança

Utilizada para reaproveitamento e especialização de classes.

8.3 Polimorfismo

Aplicado em:

- métodos sobrescritos (`calcularXP`)

- uso de interface (Promocional)
- manipulação genérica de produtos

8.4 Abstração

Classes como Produto e Cliente são abstratas, representando conceitos genéricos.

8.5 Interfaces

Permitem extensibilidade do sistema sem modificar classes existentes.

9. Fluxo do Sistema

O funcionamento geral do sistema segue as etapas:

1. Cadastro de produtos
2. Cadastro de clientes
3. Abertura de pedido
4. Adição de itens ao pedido
5. Verificação de estoque
6. Cálculo do total
7. Aplicação de promoções
8. Finalização da venda
9. Atualização de XP e estoque

10. Conclusão

O sistema desenvolvido para a cafeteria **Byte & Brew** permite a aplicação prática dos principais conceitos de Programação Orientada a Objetos.

A solução demonstra organização modular, reutilização de código e boa separação de responsabilidades.

Além disso, o uso de herança, polimorfismo, interfaces e exceções personalizadas garante um sistema flexível, extensível e alinhado com boas práticas de desenvolvimento em Java.

